Cloud et systèmes distribués

Synthèse consolidée · Notions, examen et préparation des TP

Synthèse du support fourni (42 pages), des notes Word, de la transcription et du résumé Plaud du 23 octobre, complétée par les fiches et simulations existantes. L'enregistrement s'arrête pendant l'introduction aux systèmes HPC ; les architectures de la suite du PDF sont aussi à travailler, sans les présenter comme déjà traitées oralement.

Ce qui est établi, ce qui reste à confirmer

Annoncé oralement

L'examen comporte quelques questions de mathématiques appliquées aux systèmes distribués, et aussi des questions non mathématiques. Transcription du 23/10, 02:37:44 (p. 63).

Pratique évoquée

Machines virtuelles et équilibrage de charge sur Google Cloud. Transcription, 00:03:28–00:04:09 (p. 2). Le parcours TP ci-dessous est un entraînement proposé, pas le sujet officiel.

À confirmer

Les dates exactes, la durée définitive, les documents autorisés et le barème des évaluations. Le Word et l'oral évoquent un écrit en décembre et un final ; la pondération 50/25/25 évoquée à 00:07:35 ne constitue pas une consigne officielle stabilisée ici.

Priorités proposées

Les priorités ci-dessous reposent sur le support et l'insistance de l'oral. Elles ne prédisent ni le sujet ni la probabilité d'une question. Les suggestions de Plaud ne sont pas reprises comme devoirs donnés par le professeur.

Un parcours de révision en quatre passages

1. Comprendre : nœuds, middleware, transparence, pannes et stratégies de mise à l'échelle (p. 2–8).

2. Calculer : identifier le modèle, convertir les unités, vérifier la stabilité, interpréter les résultats (p. 9–12).

3. S'entraîner : faire le sujet blanc sans regarder les corrections (p. 13).

4. Manipuler : suivre le parcours VM / répartiteur et expliquer les observations (p. 14–15). Livre et sources : p. 16.

Ce guide complète la Fiche 01 illustrée. Dans l'atelier, les simulations existantes restent disponibles et les nouvelles rubriques Synthèse, Examen et TP reprennent le même contenu. Les apartés privés sont exclus.

Fondations du distribué

Notions vérifiées dans le support ; les compléments sont signalés.

1 · Nœuds autonomes, pannes partielles et middleware

Socle · Support p. 11–15 et 19 ; oral 01:17:44–01:22:14 et 01:32:58–01:34:42

Un système distribué fait coopérer des nœuds autonomes pour fournir un service cohérent. Un nœud peut être une machine ou un processus. Les machines conservent leurs OS locaux et leurs horloges.

Un système physiquement distribué peut garder un contrôle central. La décentralisation concerne aussi l'organisation du contrôle ; multiplier les machines ne suffit pas à la démontrer.

Le middleware se place entre les applications et les OS locaux : RPC, messagerie, nommage ou sécurité selon le produit. Ses instances coopèrent à travers le réseau. L'expression « OS des systèmes distribués » est une analogie.

Une panne peut ne toucher qu'un composant. Un timeout indique l'absence de réponse à temps, sans prouver que l'opération n'a pas été exécutée. Réessayer une écriture peut donc la dupliquer.

Piège : Ne pas supposer un réseau fiable, sûr, homogène ou statique ; ni une latence nulle, une bande passante infinie, des transferts gratuits ou une administration unique.

À savoir faire : Dessine deux machines avec application → middleware → OS → matériel, puis montre le trajet d'un appel distant.

2 · Objectifs, transparence et ouverture

Socle · Support p. 13–15 ; oral : objectifs et formes de transparence

Quatre objectifs : partager les ressources, masquer les détails utiles à masquer, permettre l'interopérabilité et passer à l'échelle.

Sept transparences : accès (formats/méthodes), localisation (où), migration (déplacement), relocalisation (déplacement pendant l'usage), réplication (copies), concurrence (accès simultanés), panne (défaillance/reprise).

Masquer l'existence de copies ne garantit pas leur cohérence immédiate. Les garanties de cohérence dépendent du protocole choisi et du coût de coordination accepté.

Ouverture : interfaces publiées et compatibles, portabilité et remplacement de composants. Séparer politique (objectif/règle, par exemple durée de validité du cache) et mécanisme (moyen, par exemple horodatage et invalidation). Ouvert ne veut pas dire sans authentification.

Piège : La transparence a un coût et des limites : une application peut avoir besoin d'exposer l'emplacement, la latence ou un échec.

À savoir faire : Pour chaque forme de transparence, donne un exemple concret et une limite.

Tenue en charge et files

Notions vérifiées dans le support ; les compléments sont signalés.

3 · Évolutivité et gestion des copies

Socle · Support p. 15–18 ; oral 02:34:34–02:36:26

Trois dimensions : taille (utilisateurs/nœuds), géographie (distance/latence) et administration (organisations/règles). L'extensibilité ajoute des fonctions ; elle ne se confond pas avec la tenue en charge.

Verticale : augmenter CPU/RAM/stockage d'une machine. Horizontale : ajouter des machines et répartir le travail. Ni l'une ni l'autre ne supprime automatiquement le goulot de la base ou du réseau.

Partitionner répartit des données différentes ; répliquer crée plusieurs copies ; un cache conserve une copie pour accélérer les accès. Le partitionnement seul n'ajoute pas de redondance.

L'asynchronisme masque une attente et libère des ressources ; il ne rend pas le réseau instantané. Déporter du calcul côté client peut réduire la charge, mais les contrôles de sécurité restent nécessaires côté serveur.

Piège : Distribution verticale des couches (support p. 34) et mise à l'échelle verticale d'une machine sont deux notions différentes. Un répartiteur ne synchronise pas les données.

À savoir faire : Diagnostique d'abord CPU, stockage ou réseau ; propose ensuite une solution et explique ce qu'elle coûte.

4 · File M/M/1 : comprendre avant de calculer

Maths annoncées ; modèle central du support · Support p. 16–17 ; oral 02:06:28–02:23 environ

Hypothèses : arrivées Poisson de taux λ, services exponentiels indépendants de taux μ, un serveur, file de capacité illimitée. Régime stationnaire stable : $\lambda < \mu$. $S = 1/\mu$ est le traitement moyen, R inclut l'attente.

$U = \lambda/\mu$; $p_k = (1 - U)U^k$ pour $k \geq 0$; $p_0 = 1 - U$; $L = U/(1 - U)$. L est le nombre moyen dans le système, attente ET service, noté N dans le cours.

$R = 1/(\mu - \lambda) = S/(1 - U)$. Attente $W = R - S$. Loi de Little : $L = \lambda R$ avec des moyennes stables et le même périmètre ; cette loi n'exige pas, à elle seule, un modèle M/M/1.

Exemple : $\lambda = 80$ req/s, $S = 10$ ms $= 0{,}010$ s, $\mu = 100$ req/s. $U = 0{,}8$; $R = 50$ ms ; $W = 40$ ms ; $L = 4$; $p_0 = 20$ % ; $p_2 = 0{,}2 \times 0{,}8^2 = 12{,}8$ %.

Piège : À $\lambda \geq \mu$, ne pas appliquer les formules stationnaires. Le serveur ne s'arrête pas instantanément : la file n'a plus de régime stationnaire stable dans ce modèle.

À savoir faire : Refais la dérivation : $\lambda p_k = \mu p_{k+1}$, puis somme géométrique $\Sigma p_k = 1$, puis Little.

Cloud et architectures

Notions vérifiées dans le support ; les compléments sont signalés.

5 · ASP, cloud, cluster et grid

Socle · Support p. 8–10 et 19–22

ASP = Application Service Provider : prestataire qui héberge une application et en loue l'usage. Les ASP historiques précèdent les offres SaaS modernes ; un accès navigateur n'est pas la seule forme possible.

IaaS : ressources d'infrastructure, avec OS et application à exploiter côté client. PaaS : plateforme gérée pour déployer du code. SaaS : application prête à utiliser. La répartition précise dépend du service/contrat ; accès et données demandent toujours une gestion côté client.

Cluster : ensemble de machines souvent homogènes sous une administration commune, réseau local rapide. Grid : ressources plus hétérogènes, souvent réparties entre organisations. Cloud : ressources accessibles comme services ; ce terme ne signifie pas simplement beaucoup de serveurs.

Une migration doit considérer le coût du calcul ET des transferts, la latence, les données, les contraintes et la disponibilité. Une solution moins chère par requête peut coûter davantage à cause des échanges.

Piège : Ne pas traiter les chiffres matériels donnés comme exemples comme des définitions universelles des clusters.

À savoir faire : Pour une VM, un environnement de déploiement géré et une messagerie prête à utiliser, identifie l'offre et ce que le client administre.

6 · Couches, services et messages

Suite du PDF à préparer · Support p. 23–24 et 28–33 ; complément HTTP RFC 9110

Couche = séparation logique des responsabilités ; tiers = déploiement sur une machine/processus distinct. Présentation, logique métier et données peuvent être trois couches sur une machine ou réparties sur plusieurs tiers.

RPC : appeler une opération distante avec une abstraction d'appel. Il reste des latences, des échecs et des résultats incertains. MOM : échanges via une infrastructure de messages ; persistance et livraison dépendent du produit et de sa configuration.

REST organise l'accès à des ressources par des représentations et une interface uniforme. Stateless signifie que le serveur ne dépend pas d'un contexte de session client conservé entre requêtes ; il peut conserver les données métier.

GET lit ; PUT crée ou remplace l'état de la ressource cible ; POST demande un traitement selon la ressource cible. Ne pas mémoriser la simplification « PUT = créer, POST = modifier ».

Publication/abonnement : producteurs et consommateurs passent par des sujets/événements. Le découplage dans l'espace et dans le temps dépend du système ; une livraison durable exige des mécanismes adaptés. Linda : out écrit un tuple, in retire un tuple correspondant, rd le lit sans le retirer.

Piège : Dans le modèle du support : $N(N-1)$ adaptateurs orientés entre N applications, contre $2N$ via un intermédiaire. Ce n'est pas un comptage de câbles physiques.

À savoir faire : Compare un appel RPC bloquant et un message asynchrone pour traiter une commande.

P2P, transactions et énergie

Notions vérifiées dans le support ; les compléments sont signalés.

7 · P2P, Chord et recherches

Suite du PDF à préparer · Support p. 34–39

P2P structuré : règles de placement et de routage (DHT). Dans Chord, une clé est portée par son successeur : premier nœud actif dans le sens horaire atteignant ou dépassant la clé, avec retour à zéro.

Exemple du support : identifiants sur 5 bits, donc 32 positions possibles, mais 9 nœuds actifs. Clé 3 depuis 9 : 9 → 28 → 1 → 4. Le nœud 4 porte la clé. Clé 30 : responsable 1 après bouclage.

Les fingers utilisent des départs $n + 2^i$ modulo 2^m puis leurs successeurs. On choisit un raccourci précédant la cible ; un saut final rejoint son responsable.

Non structuré : inondation ou marche aléatoire. Avec sondages uniformes indépendants et r copies sur N nœuds, E[K] ≈ N/r. Ce nombre est une moyenne, pas une garantie ni nécessairement une mesure de réseau réel.

Superpairs, edge et distribution de fragments sont d'autres organisations. Les exemples historiques du support illustrent des principes, pas nécessairement le fonctionnement actuel des produits cités.

Piège : Une couche overlay est un graphe logique. La formule d(d − 1)^(k − 1) décrit une frontière idéale d'inondation, pas un total cumulé ni un réseau réel avec boucles.

À savoir faire : Explique le responsable d'une clé, puis justifie chaque saut sans confondre identifiant possible et nœud actif.

8 · Transactions, mobilité et énergie

Suite du PDF à préparer · Support p. 23–26

Une transaction regroupe des opérations avec une décision de validation ou d'abandon. L'atomicité impose tout ou rien ; envoyer plusieurs appels réseau ne suffit pas à fournir cette propriété.

Les systèmes mobiles et capteurs doivent gérer déconnexions, ressources limitées et consommation. Pour un capteur : τ = T_actif/(T_actif + T_sommeil). Avec 2 s actives et 8 s de sommeil, τ = 20 %.

Le mode hors ligne d'une PWA nécessite une stratégie de cache et de synchronisation. Un service worker ne résout pas automatiquement les conflits lors de la reconnexion.

Piège : Un pourcentage de temps actif n'est pas directement un pourcentage d'énergie économisée : il faut aussi connaître les puissances et coûts de transition.

À savoir faire : Décris ce qui arrive à une écriture faite hors ligne si la même donnée a été modifiée ailleurs.

Objets, services et conteneurs

Notions vérifiées dans le support ; les compléments sont signalés.

9 · Architectures orientées objet et services

Support p. 29 ; comparaison SOA : complément pédagogique · Support p. 29, section 2.1.2 ; synthèse et exemple proposés

Orientée objet : les composants encapsulent un état et exposent des méthodes. Exemple : un objet Panier contient des lignes et propose ajouterProduit(). Son implémentation reste cachée. Il peut être local ; s'il est distant, un proxy et un middleware peuvent transporter l'appel sur le réseau.

Un appel de méthode distant ressemble à un appel local dans le code, mais ajoute sérialisation, latence et pannes partielles. Un timeout ne prouve pas que la méthode n'a pas modifié l'objet. Ni les classes ni l'encapsulation ne suppriment les contraintes du distribué.

Orientée services (SOA) : organiser des capacités métier derrière des contrats explicites. Exemple : un service Paiement propose une opération autoriserPaiement(), utilisée par plusieurs applications sans exposer son code interne. HTTP/REST, RPC ou des messages sont des moyens de communication possibles.

Les deux approches sont compatibles : un service peut être développé avec des objets. Objet décrit surtout la structure du composant et son interface ; service décrit une capacité accessible par contrat. Une architecture SOA n'impose ni un protocole unique ni un bus central obligatoire.

Piège : Un objet distant n'est pas automatiquement un microservice : il faut aussi considérer le découpage métier et l'autonomie de déploiement.

À savoir faire : Compare Panier.ajouterProduit() et le contrat d'un service Commandes ; explique ce qui change si l'appel traverse le réseau.

10 · Microservices et Docker : deux rôles différents

Complément pédagogique ; pas un sujet d'examen confirmé · Documentation officielle Docker et Microsoft Azure Architecture Center ; sources en fin de guide

Microservices : découper une application par capacités métier autonomes, déployables indépendamment. Dans une boutique : Catalogue, Commandes, Paiement. On peut multiplier les instances du seul Catalogue. Chaque service maîtrise ses données et ses contrats ; une base physique distincte par service n'est pas une obligation universelle.

Le gain d'autonomie demande de gérer réseau, timeouts, sécurité et diagnostic entre services. Si le paiement échoue après la création d'une commande, l'ensemble n'est pas automatiquement annulé : il faut prévoir états, reprise ou compensation. Un monolithe modulaire peut rester un meilleur choix pour une petite application.

Docker sert à construire et exécuter des conteneurs. Image : modèle de fichiers et configuration ; conteneur : processus isolé créé depuis une image. Dockerfile : recette de construction ; registre : stockage des images ; volume : données conservées indépendamment du conteneur. Compose décrit et lance plusieurs services conteneurisés.

Une VM possède son noyau invité ; les conteneurs Linux partagent le noyau de l'hôte Linux. Sur Windows, Docker Desktop peut fournir cet hôte via une VM ou WSL 2. Plusieurs conteneurs sur un même hôte ne protègent pas d'une panne de cet hôte.

Piège : Docker n'est pas une architecture microservices : on peut conteneuriser un monolithe et exécuter des microservices sans Docker. Compose ne transforme pas un projet en cluster tolérant aux pannes.

À savoir faire : Dessine Client → Catalogue / Commandes → Paiement ; place un conteneur par service, puis montre ce qui reste partagé sur l'hôte.

Événements et données partagées

Notions vérifiées dans le support ; les compléments sont signalés.

11 · Événements, publish/subscribe et espaces partagés

Support p. 30–31 ; point signalé pendant la séance du 24 octobre · Support p. 30–31, sections 2.1.4.1 à 2.1.4.3 ; récap Plaud de la séance à intégrer dès réception

Modèle du cours	Temporel	Référentiel
Direct	Couplé	Couplé
Événement éphémère	Couplé	Découplé
Boîte persistante adressée	Découplé	Couplé
Espace de tuples conservés	Découplé	Découplé

Un événement décrit un fait survenu, par exemple CommandeCréée. Dans publish/subscribe, un producteur publie sur un sujet (topic) ou un bus ; les abonnés intéressés reçoivent une notification. Exemple : Email et Stock réagissent à la même commande sans que le service Commandes connaisse chacun de leurs destinataires.

Couplage référentiel : faut-il désigner le destinataire pour communiquer ? Le découplage référentiel permet de passer par un sujet ou de chercher une donnée par son contenu. Couplage temporel : les participants doivent-ils être disponibles en même temps ? Le découplage temporel demande qu'un intermédiaire conserve ce qui doit être récupéré plus tard.

La grille du cours compare quatre modèles : échange direct = couplage temporel et référentiel ; événement éphémère = couplage temporel, découplage référentiel ; boîte aux lettres persistante adressée = découplage temporel, couplage référentiel ; espace de données partagé avec conservation des tuples = découplage temporel et référentiel.

Nuance : publish/subscribe n'implique pas automatiquement de perdre les événements hors ligne, ni de pouvoir les rejouer. Cela dépend de la conservation, des abonnements et des garanties du système. Redis Pub/Sub classique, par exemple, ne rejoue pas les messages manqués après une déconnexion. Un mécanisme durable configuré pour conserver les messages peut fournir un découplage temporel.

Dans un espace de données partagé de type Linda, un producteur dépose un tuple, par exemple (commande, 42, prête). Le consommateur cherche un tuple correspondant à un motif, sans identifier son producteur. Le support précise : out(t) ajoute un tuple ; rd(motif) lit une copie sans retirer le tuple ; in(motif) récupère et retire un tuple correspondant. in et rd attendent si aucun tuple ne correspond ; deux out identiques créent deux occurrences.

Lien avec les microservices : un service Commandes peut publier CommandeCréée ; Stock et Email réagissent indépendamment. Publication ne signifie pas que tous ont déjà traité l'événement. Réessais, doublons et ordre dépendent des garanties ; identifier les événements et rendre certains traitements idempotents aide à éviter de répéter un effet.

Piège : Asynchrone ne signifie pas durable. Un espace de tuples n'est ni une mémoire RAM commune à tous les OS ni une diffusion automatique vers tous les abonnés : in et rd sélectionnent des tuples par motif.

À savoir faire : Classe un appel direct, un topic éphémère, une boîte aux lettres persistante et Linda selon les deux couplages. Puis explique ce qui arrive si le consommateur est absent lors de la publication.

Linda, en clair

Support p. 31–32 · Définition, opérations et exercice corrigé

Linda est un langage de coordination : plusieurs processus communiquent et se synchronisent grâce à un espace de tuples partagé. On peut ajouter ces opérations à un langage comme C. Imagine un tableau commun : un processus dépose une information, un autre la cherche par son contenu, sans connaître son producteur. Ce tableau est une abstraction logicielle, pas une RAM commune aux OS.

Un tuple est une suite ordonnée de valeurs, par exemple ("commande", 42, "prête"). Un motif précise les valeurs recherchées ; ?x indique une valeur à récupérer dans la variable x. La syntaxe ci-dessous est un pseudo-code proche de celle du cours.

Les trois opérations

Opération	Effet	Attente
out(t)	Dépose une nouvelle occurrence du tuple.	N’attend pas qu’un consommateur le lise.
rd(motif)	Lit une copie, sans retirer le tuple.	Attend si aucun tuple ne correspond au motif.
in(motif)	Récupère et retire une occurrence correspondante.	Attend si aucun tuple ne correspond au motif.

Attention au nom : out dépose dans l’espace ; in prend dans l’espace. rd lit sans retirer.

Exercice corrigé · espace initialement vide

A et B sont deux processus. Les lignes ci-dessous suivent un ordre proposé ; B peut avancer pendant que A attend.

Instruction	Résultat	Espace après
A : out("note", 12)	Le tuple est déposé.	("note", 12)
A : rd("note", ?x)	x = 12 ; la lecture conserve le tuple.	("note", 12)
A : in("note", ?y)	y = 12 ; le tuple est retiré.	Vide
A : in("note", ?z)	A se bloque : aucun tuple correspondant.	Vide
B : out("note", 18)	Un autre processus dépose le tuple ; la demande de A reprend avec z = 18 et le retire.	Vide après la reprise de A

Raisonner pour un exercice

Tracer l’espace après chaque instruction : ajout, conservation ou retrait. Une opération bloquée empêche la suite de ce processus d’avancer, mais pas les autres processus.

La présence d’un tuple quelconque ne suffit pas : le motif doit correspondre. out("autre", 18) ne débloque pas in("note", ?z).

Deux out identiques donnent deux occurrences. Une même occurrence ne peut pas être retirée deux fois, même si deux consommateurs sont en concurrence.

Avec plusieurs tuples correspondants ou plusieurs processus, ne pas inventer un ordre FIFO : considérer les choix et ordres d’exécution possibles, sauf indication contraire de l’énoncé.

Le support propose les philosophes à table et un échange client/serveur, pages 31–32. Dans un modèle à jetons, in prend une ressource et out la remet à disposition. Savoir repérer les attentes et les blocages est utile pour ces exercices.

Support p. 31–32 ; exercice numérique proposé pour l’entraînement. La possibilité d’un exercice Linda est signalée par Arthur à partir de l’oral ; le sujet exact n’est pas connu.

Les formules et leurs limites

Support p. 16–17, 26 et 36 ; dimensionnement : application proposée

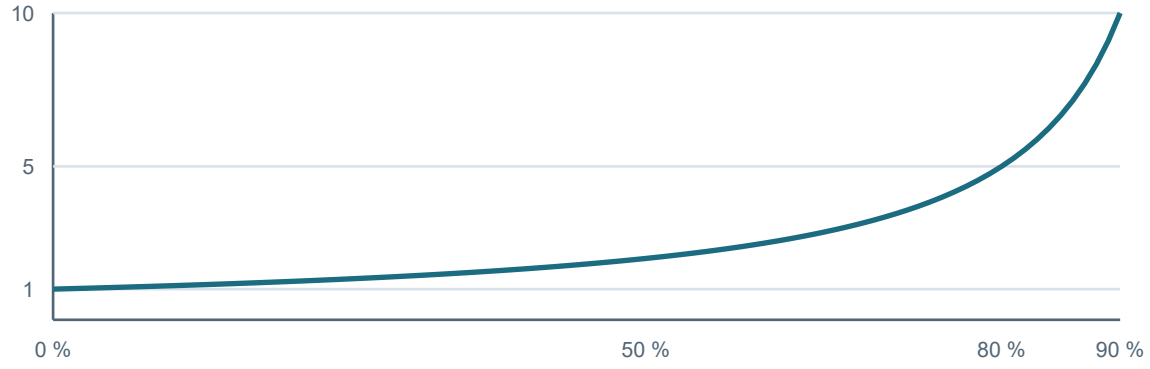
Calcul	Formule	Conditions
Capacité et utilisation	$\mu = 1/S$; $U = \lambda/\mu$	S en secondes si μ est en req/s.
Probabilités M/M/1	$p_k = (1 - U)U^k$; $p_0 = 1 - U$	k entier ≥ 0 ; régime stable.
Nombre et réponse	$L = U/(1 - U)$; $R = 1/(\mu - \lambda)$; $W = R - S$	L compte attente et service ; R, W, S en secondes.
Little	$L = \lambda R$	Moyennes stables, même système et mêmes unités.
Machines idéales	$n_{min} = \text{plafond}[\lambda/(U_{max}\,\mu)]$	$n \geq 1$; $0 < U_{max} < 1$; charge uniforme, serveurs identiques.
Après une panne	$U = \lambda/((n - 1)\mu)$	$n > 1$; même demande répartie sur les survivants ; vérifier $U < 1$.
Copies / capteur	$E[K] \approx N/r$; $\tau = T_{actif}/(T_{actif} + T_{sommeil})$	Recherche indépendante uniforme ; cycle de service ≠ énergie exacte.

Retrouver la loi stationnaire

$\lambda p_k = \mu p_{k+1}$ donc $p_k = U^k p_0$. Pour $U < 1$: $\Sigma\, U^k = 1/(1 - U)$. Comme $\Sigma\, p_k = 1$, $p_0 = 1 - U$. Le nombre moyen est $L = U/(1 - U)$, puis Little donne $R = L/\lambda = 1/(\mu - \lambda)$, pour $\lambda > 0$. À demande nulle, le temps d'une requête isolée tend vers S.

La proximité de la saturation

Rapport R/S (×) ; abscisse : utilisation U



À 80 % : R = 5S ; à 90 % : R = 10S ; à 99 % : R = 100S (hors graphique). Il faut une marge de capacité, pas seulement une égalité entre arrivées et capacité.

Verticale et horizontale : ne pas comparer sans hypothèses

Si la capacité verticale est multipliée par f : $R = 1/(f\mu - \lambda)$. Pour n files M/M/1 séparées avec répartition uniforme : $R = 1/(\mu - \lambda/n)$. Ces modèles idéaux négligent réseau, coordination et goulots communs ; ils ne décrivent pas une file unique M/M/n.

Questions corrigées · 1/3

Entraînements proposés ; aucune question précise annoncée comme sujet officiel

1 · Pourquoi un timeout ne prouve-t-il pas l’échec d’une écriture ?

Concepts · Support p. 14 et 19 ; oral 01:32:58–01:34:42

La demande peut avoir été traitée puis la réponse perdue ou retardée. Avant de réessayer, utiliser selon le besoin un identifiant de requête, une opération idempotente ou un mécanisme de déduplication. Un timeout seul ne distingue pas lenteur, coupure et panne.

2 · Le middleware remplace-t-il les OS ? Dessine ses couches.

Concepts · Support p. 13

Non. Sur chaque machine : application → middleware → OS local → matériel. Les instances coopèrent à travers les OS et le réseau. Elles fournissent par exemple RPC ou messagerie.

3 · Deux bases contiennent des données différentes. Est-ce une réplication ?

Concepts · Support p. 18

Si elles se partagent des sous-ensembles distincts, c’est un partitionnement. Si chacune contient une copie d’une même donnée, c’est de la réplication. On peut combiner les deux en répliquant chaque partition.

4 · λ = 80 req/s, S = 10 ms : calculer μ, U, R, W, L et p_2.

Maths · Support p. 16–17

S = 0,010 s ; μ = 100 req/s ; 80 < 100 donc stable. U = 0,8 ; R = 1/20 s = 50 ms ; W = 50 − 10 = 40 ms ; L = 80 × 0,05 = 4 ; $p_2 = 0{,}2 \times 0{,}8^2 = 0{,}128 = 12{,}8$ %. L inclut le service.

Méthode : rédiger une réponse sans aide, puis vérifier le raisonnement et les unités. Les corrections sont masquées au départ dans l’application.

Questions corrigées · 2/3

Entraînements proposés ; aucune question précise annoncée comme sujet officiel

5 · Comment retrouver p_k sans apprendre sa formule seule ?

Maths · Support p. 16 ; démonstration orale

L'équilibre local donne $\lambda p_k = \mu p_{k+1}$, donc $p_k = U^k p_0$. Pour $U < 1$, $\Sigma U^k = 1/(1 - U)$. La normalisation $\Sigma p_k = 1$ impose $p_0 = 1 - U$. Ainsi $p_k = (1 - U)U^k$.

6 · λ = 500 req/s, μ = 100 req/s par machine, U ≤ 80 % : combien de machines ? Et pour tenir le même objectif après une panne ?

Maths · Application proposée des p. 15–18

Charge uniforme : $n \geq 500/(0{,}8 \times 100) = 6{,}25$, donc 7 machines. Pour garder cet objectif après la perte d'une machine, il faut $n - 1 \geq 6{,}25$, donc 8 au départ. Cette marge est un exercice proposé, sous ces seules hypothèses.

7 · À μ = 100 req/s, passer de λ = 80 à 95 req/s multiplie R par combien ?

Maths · Support p. 16–17

$R_1 = 1/(100 - 80) = 0{,}05$ s. $R_2 = 1/(100 - 95) = 0{,}20$ s. Le temps de réponse est multiplié par 4, alors que les arrivées n'augmentent que de 18,75 %. La proximité de saturation explique cet effet non linéaire.

8 · Trois couches imposent-elles trois machines ? Que signifie stateless ?

Architectures · Support p. 28–31

Non : les couches sont logiques et les tiers décrivent le déploiement. Stateless signifie que chaque requête apporte le contexte nécessaire au traitement sans dépendre d'une session client stockée entre requêtes ; cela n'interdit pas une base de données métier.

Méthode : rédiger une réponse sans aide, puis vérifier le raisonnement et les unités. Les corrections sont masquées au départ dans l'application.

Questions corrigées · 3/3

Entraînements proposés ; aucune question précise annoncée comme sujet officiel

9 · Dans l’anneau du cours, qui porte les clés 3, 12 et 30 ?

Architectures · Support p. 34–35

Nœuds actifs : 1, 4, 9, 11, 14, 18, 20, 21, 28. Les responsables sont 4, 14 et 1. Pour 30, on dépasse le dernier nœud puis on revient au premier. Il n’y a pas un serveur pour chaque identifiant.

10 · 10 000 nœuds, 10 copies : combien de sondages en moyenne ? Capteur actif 2 s puis endormi 8 s : quel cycle ?

Maths · Support p. 26 et 36

Sondages indépendants uniformes : $E[K] \approx 10\,000/10 = 1\,000$, sans garantie de maximum. Cycle actif : $\tau = 2/(2 + 8) = 0{,}2 = 20\ \%$. Ne pas diviser uniquement par le temps de sommeil.

11 · Ajouter un load balancer double-t-il la capacité et synchronise-t-il les données ?

TP · Entraînement lié au TP évoqué ; support p. 18

Non. Il distribue les requêtes vers des backends. Le gain dépend de leur capacité, de la répartition et des goulots partagés. Les données, sessions et écritures ont besoin de mécanismes spécifiques ; les health checks détectent une indisponibilité avec un délai.

12 · Deux VM servent des pages différentes après équilibrage. Comment diagnostiquer ?

TP · Entraînement proposé VM et load balancing

Afficher l’identité/version de chaque backend, vérifier le déploiement et la configuration, examiner les logs et le cache. Répéter des requêtes n’impose pas une alternance stricte ; l’affinité et la politique du répartiteur peuvent maintenir un même backend.

Méthode : rédiger une réponse sans aide, puis vérifier le raisonnement et les unités. Les corrections sont masquées au départ dans l’application.

Sujet blanc · 45 minutes

Barème d'entraînement proposé /20 ; durée et répartition non officielles

Prévoir papier et calculatrice pour l'entraînement. Vérifier auprès de l'enseignant le matériel réellement autorisé à l'examen. Écrire les hypothèses, montrer les conversions, puis commenter les résultats.

A · Expliquer — 5 points

Dessiner les couches de deux nœuds, situer le middleware, expliquer une panne partielle et deux transparences avec leurs limites.

B · Calculer — 7 points

Pour λ = 80 req/s et S = 10 ms, établir la stabilité, calculer U, R, W, L, p_2. Puis calculer R pour λ = 95 req/s et interpréter.

C · Dimensionner — 4 points

À λ = 500 req/s et μ = 100 req/s par VM, viser U ≤ 80 %. Dimensionner avec puis sans la perte d'une VM. Donner deux limites du modèle.

D · Architecturer et diagnostiquer — 4 points

Proposer un client → répartiteur → deux VM → stockage partagé. Distinguer répartition et réplication. Expliquer quoi mesurer lors d'une panne et retrouver le responsable de la clé 30 dans Chord.

Correction et autoévaluation

A · Expliquer — 5 points : Schéma correct 2 pts ; panne et incertitude 1 pt ; transparences et limites 2 pts.

B · Calculer — 7 points : Unités et stabilité 1 pt ; U/R/W 2 pts ; L/p_2 2 pts ; comparaison et interprétation 2 pts. Correction : 80 %, 50 ms, 40 ms, 4, 12,8 %, puis 200 ms (×4).

C · Dimensionner — 4 points : 7 VM sans panne 1 pt ; 8 VM pour tenir l'objectif après une panne 1 pt ; deux limites argumentées 2 pts : déséquilibre, coûts réseau, base commune, détection de panne…

D · Architecturer et diagnostiquer — 4 points : Architecture et données 1 pt ; mesures/health checks 2 pts ; responsable 1 dans Chord 1 pt.

Pour A : application → middleware → OS → matériel sur chaque machine ; panne partielle et incertitude d'un timeout ; deux transparences illustrées avec leurs limites. Pour D : le répartiteur oriente les requêtes, la réplication duplique des données ; observer erreurs, latence et état des backends avant/pendant/après la panne.

Avant de rendre

Résultats avec unités ? Stabilité contrôlée ? L compte-t-il bien l'attente et le traitement ? Arrondi du nombre de machines au supérieur ? Solution architecturale justifiée par un goulot identifié ? Les limites du modèle sont-elles écrites ?

TP · Construire et observer

Préparation proposée du TP évoqué à l’oral : machines virtuelles et load balancing sur Google Cloud



B arrêté ≠ B détecté ; après retrait de B, A doit absorber toute la demande.

Le répartiteur ne réplique ni les données ni les sessions des VM.

1 · Préparer et dessiner

Reprendre l’environnement et le type de répartiteur indiqués par l’enseignant. Dessiner client → frontend du répartiteur → backends VM ; ajouter réseau, port de service et health check. Pour une réalisation cloud, vérifier projet, droits, quotas et coût avant création.

Preuve attendue : Schéma légendé, rôle de chaque composant et liste des ressources prévues.

Question possible : Pourquoi un load balancer n’est-il pas une base répliquée ?

2 · Servir une page sur une VM

Créer une VM selon les consignes du TP, installer/démarrer un service HTTP et afficher un identifiant de backend dans la réponse. Vérifier service local puis accès réseau autorisé ; distinguer port d’écoute, routage et pare-feu.

Preuve attendue : Réponse HTTP et identité du backend ; configuration de port et règle d’accès pertinentes.

Question possible : Le service fonctionne localement mais pas à distance : quelles couches vérifier ?

3 · Déployer plusieurs backends

Déployer le même service sur deux instances ou dans le groupe demandé. Configurer le backend service, le health check et le frontend du répartiteur suivant le guide adapté au produit retenu. Attendre que les backends soient sains.

Preuve attendue : Identités distinctes, même version applicative, état sain de chaque backend et réponse via le frontend.

Question possible : Pourquoi un backend qui répond en local peut-il rester marqué unhealthy ?

TP · Tester et expliquer

Préparation proposée du TP évoqué à l'oral : machines virtuelles et load balancing sur Google Cloud

4 · Observer la répartition

Répéter des requêtes en observant l'identité du serveur et les logs. Noter la politique/affinité éventuelle. Ne pas attendre obligatoirement une alternance A/B. Augmenter la demande uniquement dans les limites de l'environnement autorisé.

Preuve attendue : Petit tableau demande, backend, statut et latence ; décrire la méthode et ses limites.

Question possible : Quelles mesures distinguent un problème de réseau d'un serveur saturé ?

5 · Provoquer une panne contrôlée

Dans l'environnement de TP, rendre un backend indisponible, observer les health checks et refaire les requêtes. Noter le délai de détection et d'éventuelles erreurs transitoires. Avec un groupe géré, vérifier si l'instance est automatiquement recréée ; ce mécanisme est distinct du routage. Restaurer le service.

Preuve attendue : Chronologie avant/pendant/après, nombre de backends sains, erreurs et temps de réponse ; distinguer observation et hypothèse.

Question possible : Le survivant peut-il absorber toute la demande ? Calculer $\lambda/((n-1)\mu)$ dans le modèle idéal.

6 · Conclure et nettoyer

Expliquer ce qui arrive aux sessions et aux données locales lorsqu'un backend disparaît. Proposer un stockage partagé ou une réplication adaptée selon les besoins. Après le TP, supprimer les ressources créées suivant les consignes et vérifier les ressources restantes : VM, disques, IP, répartiteur et composants associés.

Preuve attendue : Conclusion sur disponibilité, cohérence et capacité ; liste des ressources supprimées/restantes. Une VM arrêtée ne signifie pas que tous les composants cessent d'être facturés.

Question possible : Pourquoi le test de panne de deux serveurs HTTP ne démontre-t-il pas la résilience de toute l'application ?

Corriger le diagnostic

Service local OK mais accès distant KO : vérifier écoute/interface, port, pare-feu et routage. Backend unhealthy : vérifier chemin/port de sonde, réponse de l'application et accès réseau des sondes. Le retrait d'un backend ne prouve pas que les sessions et données ont survécu.

À $\lambda = 160$ req/s et $\mu = 100$ req/s : deux VM idéales sont chacune à 80 %. Après perte d'une VM, le survivant serait à 160 % : aucun régime stationnaire stable. La disponibilité du processus ne suffit pas à garantir un service acceptable.

Livre et sources

Traçabilité et limites de cette consolidation

Maarten van Steen et Andrew S. Tanenbaum — Distributed Systems, 4e édition, 2023

Référence explicitement citée par le professeur p. 3 et 41. La page officielle donne accès aux diapositives et à une demande de copie numérique gratuite personnalisée. Aperçu thématique fondé sur le sommaire et les slides officiels, pas résumé d'une lecture intégrale du livre.

Livre, slides et demande de copie numérique (lien)

1 Introduction : objectifs et limites de la distribution.

2 Architectures : couches, services, événements et P2P.

3 Processus : unités d'exécution, threads et virtualisation.

4 Communication : échanges distants et messages.

5 Coordination : organisation des actions concurrentes.

6 Nommage : retrouver les entités et leurs ressources.

7 Cohérence et réplication : garanties entre plusieurs copies.

8 Tolérance aux pannes : détecter, masquer et récupérer.

9 Sécurité : authentification, accès et protection des échanges.

Commencer par les chapitres 1 et 2 correspondant au PDF fourni. Les autres chapitres sont une carte du livre à approfondir au rythme des prochaines séances, pas un programme d'examen confirmé.

Sources utilisées et corrections

Support principal — Olivier Grunder, Cloud et traitements distribués, fichier lucc01-cours01-26.pdf, 42 pages. Les références p. n sont les pages du PDF, couverture comprise.

Notes personnelles et oral — notes de cours.docx et transcription Plaud du 23/10/2026, 75 pages. Les repères hh:mm:ss renvoient à l'enregistrement. Le calendrier reste à confirmer ; les apartés privés sont exclus.

Résumé Plaud et productions précédentes — Résumé Plaud du 23/10 vérifié, Fiche 01 et Atelier Cloud. Noms corrigés : Maarten van Steen et Plouin (2022). Le récap Plaud du 24/10 n'a pas encore été reçu ; il complétera la partie événements et espaces partagés.

Livre et ressources officielles — van Steen et Tanenbaum, Distributed Systems, 4e éd., 2023 ; sommaire et diapositives officielles consultés.

HTTP : sémantique des méthodes — RFC 9110, sections 9.3.3 (POST) et 9.3.4 (PUT), pour préciser les simplifications du support.

HTTP : sémantique des méthodes (lien)

TP : création de VM — Documentation officielle Google Cloud consultée le 24/10/2026. Adapter les étapes aux consignes de l'enseignant.

TP : création de VM (lien)

TP : exemple de répartiteur — Exemple officiel de classic Application Load Balancer avec groupe géré ; exemple de préparation, pas le produit imposé pour le TP.

TP : exemple de répartiteur (lien)

Complément : conteneurs et Docker — Documentation officielle : images, conteneurs et partage du noyau. Complément au support fourni, consulté le 24/10/2026.

Complément : conteneurs et Docker (lien)

Complément : microservices — Microsoft Azure Architecture Center : autonomie métier, déploiement et compromis. Complément pédagogique, pas une annonce de programme d'examen.

Complément : microservices (lien)

Complément : durée de vie des événements — Redis Pub/Sub : messages manqués non rejoués ; les garanties dépendent du mécanisme choisi.

Complément : durée de vie des événements (lien)

Pour enrichir le cours : mettre à jour revisions.json, puis exécuter Outils/generer_revisions.py depuis Cloud/Cours. Le PDF et revision-data.js sont régénérés ensemble. Ajouter un repère de source à chaque nouvelle notion.